

Cyborg Hunter

-Proyecto Final - Informática II

1st Daniela Escobar Velandia
Dpto. de Ingeniería Electrónica y Telecomunicaciones
Universidad de Antioquia
Medellín, Colombia
daniela.escobarv@udea.edu.co

I. INTRODUCCIÓN

El presente proyecto corresponde al desarrollo de un videojuego utilizando programación orientada a objetos en C++ y el framework Qt. La aplicación fue concebida como una integración práctica de los conceptos vistos durante el curso de Informática II, incluyendo diseño orientado a objetos, herencia, polimorfismo, memoria dinámica, contenedores, interfaces gráficas y simulación de fenómenos físicos.

El videojuego debe ser interesante de jugar, tener un nivel de dificultad adecuado, amigable y agradable a la vista del usuario. Se pondrá en práctica los conocimientos de: POO, memoria dinámica y GUI.

Temática del videojuego: Tema deportivo enmarcado en un universo especial.

Deporte escogido:

Tiro al blanco, también conocido como Tiro Deportivo o Tiro de Precisión, es un deporte en el que el practicante debe disparar un arma de fuego o de aire comprimido con precisión al blanco de tiro, se utilizan rifles, pistolas o escopetas.

Universo:

Enmarcado en el universo de Cyberpunk 2077 que es un videojuego ambientado en “Night City”, una ciudad futurista. Los jugadores controlan a V, un mercenario que busca un implante único para la inmortalidad, obteniendo mejoras cibernéticas.

Título del videojuego:

Cyborg Hunter

A. Resumen

CyborgHunter es un videojuego de tiro al blanco ambientado en un universo Cyberpunk donde el protagonista es Kael, un cazarrecompensas cibernético que busca reemplazar partes de su cuerpo con mejoras robóticas, por lo que participa en un torneo ilegal de Tiro al Blanco organizado en una ciudad futurista llamada “Medallo 2077”.

En la primera prueba, denominada Campo de Tiro, Kael debe demostrar sus habilidades de precisión destruyendo cabezas de robots lanzadas por cañones automáticos. Tras superar este desafío, accede a la arena principal, donde deberá enfrentarse a El Gobernador, un robot inteligente que protege el Núcleo de la Inmortalidad y que adapta parcialmente su comportamiento a las acciones del jugador.

B. Motivación

El proyecto surgió con el propósito de desarrollar una aplicación interactiva que permitiera integrar los conocimientos adquiridos durante el curso en un entorno gráfico atractivo y dinámico. Además, se buscó incorporar conceptos de inteligencia artificial básica y simulación física para hacerlo más interesante.

II. OBJETIVOS

General

Desarrollar un videojuego interactivo en C++ utilizando Qt que integre programación orientada a objetos, simulación física e inteligencia artificial básica.

Específicos

- Diseñar una arquitectura modular.
- Implementar dos niveles con mecánicas diferenciadas.
- Incluir simulaciones físicas.
- Implementar un agente autónomo con capacidades de percepción, razonamiento, acción y aprendizaje.

III. ESPECIFICACIÓN DE REQUERIMIENTOS

Requisitos funcionales

- El sistema debe mostrar un menú principal.
- Debe permitir iniciar cualquiera de los niveles.
- Debe registrar impactos sobre los objetivos.
- Debe controlar la duración de cada partida.
- Debe administrar vida, munición y granadas.
- Debe generar ítems aleatoriamente.
- Debe implementar un enemigo con comportamiento autónomo.
- Debe reproducir efectos de sonido.

A. Requisitos no funcionales

- Interfaz gráfica intuitiva.
- Código modular y reutilizable.

IV. DISEÑO Y ARQUITECTURA DEL SISTEMA

Se agrega un diagrama de clases de la capa lógica con el fin de visualizar la interacción de las clases utilizadas.

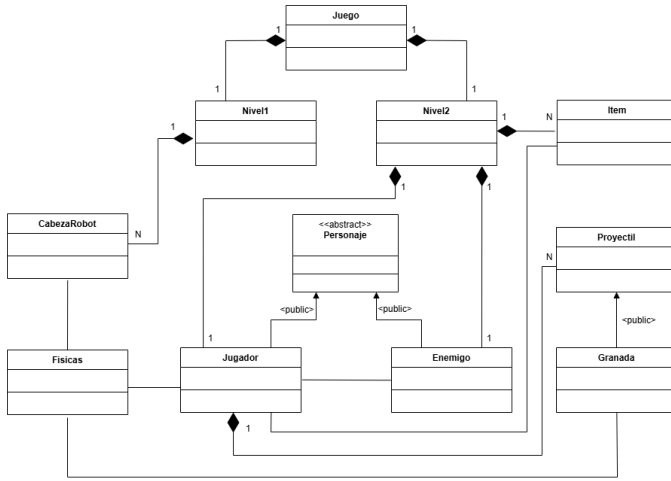


Fig. 1: Diagrama de clases.

Se gestionan de la siguiente manera:

- **Clase Juego:** Es el controlador principal. Se encarga de cambiar las escenas, gestionar los menús interactivos y reproducir la música correspondiente a cada nivel.
- **Clase Nivel1:** Es la responsable del Campo de Tiro. Gestiona la generación de objetivos con ángulos aleatorios, gestiona el número de objetivos destruidos, el tiempo de juego y pantallas de victoria/derrota.
- **Clase Nivel2:** Es la responsable del modo Enfrentamiento. Gestiona al jugador, enemigo, los items aleatorios y contiene proyectiles y granadas.
- **Clase Personaje:** Es una clase abstracta creada para evitar replicar las funciones base de enemigo y jugador, como la posición, el movimiento y la vida.
- **Clase Proyectotil:** Gestiona el movimiento de los proyectiles, la velocidad y su dirección, así mismo para la Granada.
- **Clase Item:** Representa los objetos que le proporcionan al personaje recuperación de vida o activar sus características.
- **Clase CabezaRobot:** Representa los objetivos móviles del primer nivel.
- **Clase Fisicas:** Contiene las ecuaciones necesarias para hacer las simulaciones respectivas de los objetos.

Dependencias externas

Se utilizaron principalmente módulos de Qt:

- Qt Widgets
- QGraphicsScene
- QGraphicsView
- QGraphicsPixmapItem
- QTimer
- QVector
- QList

- QMediaPlayer
- QAudioOutput

Además se emplearon bibliotecas estándar de C++ para operaciones matemáticas y manejo general de datos.

V. DESARROLLO E IMPLEMENTACIÓN

A. Características del personaje

- **Veloz:** Al recolectar piezas electrónicas específicas (las azules) que van cayendo, aumenta la velocidad de disparo durante 10 segundos.
- **Sobrecargado:** A medida que recolecta piezas electrónicas específicas (las rojas) que van cayendo, su masa aumenta. Esto lo vuelve más lento al caminar.

B. Nivel 1: El Campo de Tiro

- **Contexto:** El protagonista participa en la primera prueba del torneo. Desde una cabina de tiro futurista, debe destruir cabezas de robots que siguen trayectorias parabólicas y son lanzadas desde cañones automáticos. Existen dos niveles de dificultad que modifican la velocidad y frecuencia de aparición de los objetivos.
- **Físicas utilizadas:**
 - 1) Movimiento parabólico, para las cabezas de robots lanzadas por los cañones.

C. Nivel 2: Enfrentamiento

- **Contexto:** El protagonista entra a la arena final del torneo, situada en Medallo 2077, donde debe enfrentarse El Gobernador, un robot con inteligencia que es quien nos dará el núcleo de la inmortalidad. De la parte superior de la pantalla van cayendo items que son los que le proporcionarán a nuestro jugador vida, modo sobrecarga o modo veloz. En este nivel el jugador se desplaza utilizando el teclado y puede lanzar granadas con 3 ángulos diferentes: 60, 45 y 30, para hacerle daño a nuestro enemigo, además usa una escopeta de neutrones para infligir daño. Los recursos son limitados, tiene 5 disparos y 3 granadas disponibles, pero se van regenerando cada 5 segundos.
- **Físicas utilizadas:**
 - 1) Movimiento parabólico, para las granadas.
 - 2) Choque elástico, cuando el jugador choca con el enemigo.
 - 3) Caída libre, para los items que activan las características del personaje.
 - 4) Fricción, aplicada al personaje cuando entra en modo sobrecarga.

D. Agente autónomo

Se tiene un agente autónomo sencillo que consiste en el del nivel 2, El Gobernador, quien, después de destruido será el que suministre el núcleo de inmortalidad para nuestro cazarecompensas.

- **Percepción:** Detecta la posición del jugador, disparos y si está en un modo especial.

- **Razonamiento:** Decide cuándo atacar, retroceder o cubrirse dependiendo de las características de nuestro cazarecompensas, si está en modo veloz, nuestro agente se alejará y se cubrirá, si está en modo sobrecarga le disparará.
- **Acción:** Puede moverse, disparar y activar un escudo.
- **Aprendizaje:** Dependiendo de la posición desde dónde dispare nuestro cazarecompensas, nuestro agente se desplaza a la posición contraria en y.

E. Resultados

Las funcionalidades propuestas fueron implementadas satisfactoriamente y se obtuvo un comportamiento estable durante las pruebas.

VI. RESULTADOS Y DISCUSIÓN

El videojuego desarrollado cumple los objetivos planteados inicialmente.

Se logró integrar:

- Programación orientada a objetos.
- Interfaces gráficas.
- Simulación física.
- Gestión dinámica de memoria.
- Inteligencia artificial básica.
- El aprendizaje del agente es sencillo.
- Existe un único jefe final.
- Los escenarios son estáticos.

VII. CONCLUSIONES

El proyecto permitió aplicar de forma práctica los conceptos fundamentales de la programación orientada a objetos y el desarrollo de software interactivo.

La utilización de Qt facilitó la construcción de interfaces gráficas y la administración de recursos multimedia.

La implementación del agente autónomo permitió explorar conceptos básicos de inteligencia artificial, mientras que las simulaciones físicas enriquecieron la experiencia de juego.

Finalmente, el desarrollo evidenció la importancia de la modularidad, la reutilización de código y el diseño adecuado de clases dentro de proyectos de mediana complejidad.